experimenting_with_vLLM_on_k8s...using_microshift_podman_ks

Tue, May 12, 2026 9:30AM 20:07

SUMMARY KEYWORDS

Kubernetes, microshift, podman, desktop, taser, AI solutions, CPU usage, GPU, KP cache, memory allocation, inference server, KServe, parallel requests, OpenShift, benchmark.

SPEAKERS

Speaker 4, Speaker 3, Speaker 5, Speaker 2, Speaker 1

S Speaker 1 00:00

Welcome to the Community central theater. We are really excited that you're here. Hopefully you all can hear me. If you can hear me, can you raise your hands all right, thank you so much. Up first here is going to be Alexa libera talking about experimenting with vllm on Kubernetes using micro shift, pod man, desktop and taser. Go ahead, put your stuff up. I'll give you a second to do that if you cannot serve me. And I'm talking a little microphone I know all about you can we're going to use the power button cycle to be yellow or light green color. But on that note, go ahead, awesome.

S Speaker 2 00:44

Take it away. Hear me, good, awesome. I'll probably on the screen. So let's get started. My name is Alexis, technical mentor, and I've been working with most of our home Street and community projects. Of our home Street and community projects library can serve. But the purpose of this talk is to show you how you can task PLM on your own laptop, your own chains without using GPU at all. But before we get started, let me ask you this, how many of you here try to remember llms on your own laptop using CPUs, and you thought your machine was certainly like a plane about to take off, right? Your fan is like high enough, and that's a problem. When you're running llms on your own laptops, you don't have enough resources, right? So how can you test your AI solutions or your AI applications before you go to production laptop without wasting any of your resources and maximizing the use of your own CPU and brand? That's the purpose of this talk. I'll show you that. But as I mentioned before, you that. But as I mentioned before, one of the problems that we have is that running llms called small machine power machines consume huge resources. So if you're running with GPU, for example, you have problems with ramp. So I don't know if you face that, I face that multiple times right around LLM even with GPU or green on your laptop and heating like out of memory. That's basic, right? So that's one of the problems. And most of us here don't have enough money to acquire, like, resemble 800 because they're expensive. Another problem that we have is the complexity that involves. Because I don't know if you guys

S Speaker 3 02:42

are aware

Speaker 2 02:42

of the KP cache. The KP cache is somewhat like a way to manage all of the pieces that involves your LLM into blocks into your memory. It's a key value memory caching technology. But the problem is, for every session you need that in order to run your parents and you contest on your model, and that's also expensive to your hardware. And finally, we have this given track depending on the model that you choose. For example, if it's a huge model, it doesn't need to be a huge model, but like a somewhat bigger model, you have an issue trying to load that model every time, every time you scale down and try to scale up your application, okay, it takes some time to do that. Like, for example, in my test, it took me, like, from 20 to 15 minutes to load the model, like Gemma, for when, for example. So by the time you're modeling this up your user aligned doing something else, because it's so these are the main problems that we have. So what I'm going to present to you today is a stack that you can use in your machine not to run something that has speed. That's not the purpose of it. I'm going to explain that, but the purpose is to show you how you can test this integration, because I don't know how you are using open AI in your first environment. None of you try it. We'll talk about it, right? But the idea is that you can test PLM, which is a inference server, in your own box, without using CPU, but by leveraging the power of botton desktop, which is going to help you to deploy microchip. Microchip is lightweight version of OpenShift and installing K serve, which is the platform to use the server right using API compatible stuff, while using PLM. It has multiple benefits. Three of the main benefits that we have by green and PLM, first of all, one of the problems that we have when we try and run on it doesn't matter, because you can use it in order. The problem is the way they allocate memory. It's not the best way, right? It's somewhat like someone playing hat race the best way. You have all these gaps, all of these spaces unused. It's not contiguous. So the problem with that is that you lose performance. So pays attention trying to use the same capabilities that we have with overexisting life virtual memory in order to allocate continuous space on your memory and make your occurs faster. That's the idea. Continuous batching walks along with the pace detention in a way that, since you now have this continuous memory being allocated, you can you'll need a way for all of these requests to be batched in order to process them. They're processing as soon as they come to your model. And finally, performance test the PLM, for example, with larger models, you will see that compared personally to all llama or llama CPP, or whatever other inference server you use, you probably get like 24 plus higher performance so as I've explained before, the pace detention is the highlight here, because it's like a tattoos, right? As you can see, for example, with memory, you're wasting space. You're

Speaker 4 06:16

wasting

Speaker 2 06:18

resources. So with pace detention, that's the same as OS does you have virtual memory, memory pages been allocated continuously. And you can use this to reach a result of near zero waste and higher cures. And you see that happening even more when you rather parallel classes, right? Not serialized. I'll show you that later. And why K serve for this platform, for this stack, because k serve provides you a instantarex protocol all of your models, also, if you need, like for example, to change your model. If you're running now and now you want to change Gemma or greening or mixture of whatever else model you want to run, you don't need to recompile all of your front end or your back to rent that case, there provides that for you. Also, there are other benefits, abstract lifers and the networking is you and whatever else in order to run on top of that, it doesn't matter if it is, for example, open chip, if it is kind or if there's microchip, it doesn't matter. But also, of course, you're going to see most benefit by running on top. And the difference is proof. I updated this one by the end of my presentation, you see that there

Speaker 5 07:39

are QR code that redirected

Speaker 2 07:42

to my proposed

Speaker 4 07:43

storage,

Speaker 2 07:45

what I've been testing different models in my own laptop, this laptop, this laptop has this CPU powered, okay? It has 64 gig values of RAM and one terabyte of SSD. It's not that much, right? So all my tests I completed all this last laptop here, and you see my GitHub repository that I was able using, for example, Gemma, to reach here, somewhat like three requests per second using the yellow lab, but all lemma over power grab, outperform it. Yellow lab. Why? Because lemma or lemma are built to run on CPUs, right? So with these models, with this inference surface, I was able to get, like, four to six requests per second on CP computer. But when I went to overshoot with the LLM, then difference also, changing the model. I was tested with Java on my laptop and on top of OpenShift, somewhat bigger model, 3 billion parameters, okay, and I was able to get 8.6 request, percent. And I'm only talking about using before instances on top of AWS, it's not that much as well. So the more you get the videos model, and the more you get you use, the more you see the difference, especially if you do parallel requests, which is the base of the demo that I show you. Also you get like this, extra features, for example, or GP, as we call GP, somewhat, what it does is basically taking your model and divided it in little pieces. So if you have, like for example, multiple GPUs on the same node, it is able to bring your model into the little pieces and divide it between all the GPUs into the same node with LMD. It's almost the same concept, but the idea is to take that request and divide it, or make it high available throughout different nodes into your plus two may have like, for example, different clusters running the same, for instance, divided by all of these clusters. That's the idea behind LMD, also to be guilty of the coding. The idea behind this is to use a small model, or as we call it, draft model, to tokenize, and then use a bigger model to test or to assess these tokens. Is speeding up your inference as well. And finally, it's agnostic. You can run on top of different GPU, GPU vendors as well. And as I mentioned before, you will see most benefit by using this on top of open shop. That's the way you have to conduct. The demo that I'm going to show you, everything that we are going to do, you can do the same thing, basically the same YAML file, the same configuration, the same matrix on your production environment, right? That's why I meant test on my laptop, using the same configurations, basically, and just taking that into production. That's the so these are the models. These are the products or projects that are going to use for this piano. All right, take a picture of it and basically what we're going to do is we're going to use some scripts that I've created to facilitate our work, right? And we're going to install micro shift. We're using auto desktop on top of microservice. We're going to install K serve. K serve is going to install all of the other operators and containers that we need in order to move this operation forward. And finally, we're going to have our model available for us to run comparisons against it. This is my YAML file, my configuration. If you're running on a laptop, pay attention to this specific pair, VLM. Attention back end. Without it, you won't be able to run VLM on top of CPUs. Okay, this is mandatory, so take note of it. Use the VALUE as CPU tension, and then you read it well. As you can see, I set just four CPUs, and they gave it right for this demo. So you can see that it's possible to test even in the launch. And finally, we're going to pour forward that using these two inverse gateway and orange versus parallel versus against this water. So let's go to the demo so you can see how it works. Bounce our fingers. Now let's see if the play is going to work properly, because the time All right, so this is the dashboard of bottom and desktop. The interface of bottom desktop using bottom of desktop. What I'm going to do is to install first Microsoft, but be aware that right now it's not screwed to install microchip as rootless containers. Okay, I submitted PR for that. I'm waiting for Google. If that works, probably after Summit, you're going to be able to do so. But right now you install Microsoft using real school containers. Okay, just be aware of that. Let's move this one just a moment. Okay, we're okay? So I'll click here, and then I'll go to Microsoft, and as simple as that, I'll enable work list again. This is not available GA I hope will be by the end of Summit, and then it's installed. It's just simple as that, okay, so as you can see, I have my contest set up there, and now, using bottom and desktop desk, I can monitor I can follow up with the installation of my model. So next step, I have this repository. As you can see, before that, I'll be waiting for my chip to create all of the containers, all of the necessary containers that I need for my OpenShift or Microsoft stack to be available in order for me to move forward and install time and talk. So as soon as every container, or every default container, is ready, I'll move forward with this script one. And this is the same thing you will need to do if you're going to test this on your own machine? Okay, run the script one, and the script one is made up with a bunch of different commands for you to install the case or stack on your own question. Okay? If you're testing this with a different solution, not Microsoft, let's say, for example, you're testing on Q Kubernetes or kind of whatever else other lightweight, advanced version there are, there are other options on my Git web repository for you to check, okay, but for Microsoft specifically, this script is all you need in order to install the K serve stack. Okay, it is going to take you roughly seven to 10 minutes, all right, for our key service to be available. Of course, I speed up the view, as you can see, so you don't need to wait here, and we don't have, like that much of time. So after a few minutes, you'll see that all of the available, all of the containers, all of those containers are available, and we can move forward. I'm just checking here to make sure every container is running properly. And now we'll move forward with these commands. This script is going to generate you all this orientation for you to move forward with the demo. Okay, so first thing I need to do is to create a secret why? Because this model that I'm using is the gem model requires you a hook and paste token to have access to. So if you don't have it, please create it first. So I'm showing you here my YAML file, as I mentioned before, during this live presentation. So there are the parameters that I mentioned to you. Okay, I'll create now my stack. I've applied it, okay, and now I'll wait for a model to be available. So this is my model, Gemma three, all right? And as I mentioned before, for a model like this, it is a small model, okay? For a model like this to be up and running on my laptop only run CPUs. It took me, if I'm not mistaken, 13 minutes. Okay, with that configuration for CPUs and AP device, you can use the word to speed up the deployment. So use it on the desktop. I can follow up with it by you see the orange surface to orange circles because it deploys by default, two different containers for my model. So I can follow up with that. Just observe these two orange circles until they get green. As soon as they start with green materials might have happened in a few seconds, you'll see that the model is going to be

partially available. Okay, not 100% available. So what I suggest you do this as soon as it turns green as it is. Now, check the logs. I'll show you how the logs are, and you will see here that it shows you that the model is available. The containers are up and running, but the model is not responsible yet, as you can see by the rocks right here. So wait forward, and after a few seconds, the other container is going to be up and running. So check it now, you see that the model is up and running, but only one container out of two. As I said, after a few seconds, you see that a log showing you that like that, that your model is up before 80 is available for you to use. Okay, this one, this is what you want to see, 88 sorry, using the 88 so as soon as the containers, the two containers up and running. Now, both are up and running, you're going to run the script pre, the script pre, I built it in order to test the parallelism. Okay, so the parallelism is the benefit of the LLM. If I run a lemma, or lemma directly serialized in Paris, you will see benefit, right? So this is the test I'm running. Now, see, this is total time. This is the performance, but now I'm running benchmark. Benchmark is being run five different requests per second for a total amount of if I'm thinking I set it up as five seconds right? See the difference. The through loop is higher, and now we're running a sample of 50 requests throughout 10 seconds. Take note of the throughput of the past task, and you will see by the end of it the result. See the throughput is higher, and that's running only on CPU right? If you do the same task on GPUs, you will see that that throughput is going to be somewhat eight to 10 requests per second, and it gets higher. So that's what I wanted to show. How you can test this again, you can use the same scripts, the same YAML file on your own on shipbuster restaurant. So what's next? Then, if you want to test please Canvas key is going to redirect you to my public repository with all of these tools, with the tasks that I predicted, the benchmark that I ran on my laptop, as I mentioned before, with the script files, the gamma files, etc, and for the information on how do better with got it, and that's it. I appreciate your time. Appreciate you all coming to my presentation. I'll be around done.

S Speaker 1 20:00

If you still have your headphones on, please return them to the huge queen right behind you, and the next talk will be.